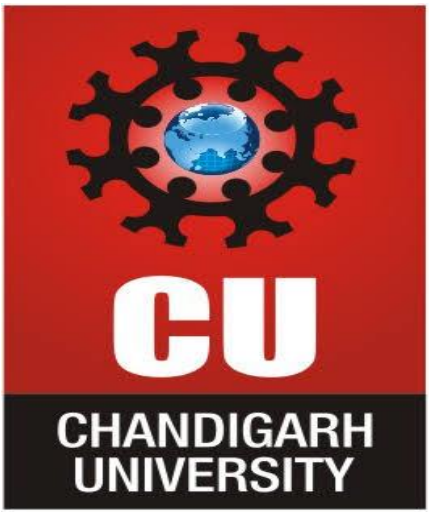
CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

DATABASE MANAGEMENT SYSTEM

Mini Project

HOTEL ROOM BOOKING SYSTEM



Submitted By	Randeep Singh
UID	25BCD10148
Submitted To	Mrs. Harkamal Kaur
Designation	Assistant Professor
Subject	Database Management System (DBMS)
Date of Submission	28 / 10 / 2026

1. Introduction

A hotel is a service business in which hundreds of small transactions take place every day: enquiries, room allocation, check-in, payment and check-out. When these are recorded in registers or scattered spreadsheets, problems such as double booking of a room, wrong billing and lost guest records become common. A Database Management System (DBMS) keeps all this information in one organised, secure and easily searchable place.

The Hotel Room Booking System is a mini project built on a relational database. It stores the details of guests, room categories, rooms, bookings and payments, and allows the hotel staff to find information such as available rooms, booking history, pending payments and revenue using SQL queries.

The project applies the main topics studied in the DBMS course: ER modelling, relational schema design, primary and foreign keys, integrity constraints, normalization, Data Definition Language (DDL), Data Manipulation Language (DML), joins, aggregate functions, sub-queries and views.

2. Objectives / Problem Statement

2.1 Problem Statement

Manual hotel booking records are slow to search, easy to duplicate and difficult to reconcile with payments. The hotel needs a reliable database that records every guest, room and booking, rejects invalid data, and can instantly answer day-to-day questions such as: Which rooms are free today? Which guests still have a pending balance? How much income did each room category generate?

2.2 Objectives

- To design a normalized relational database for hotel room reservation.
- To store guest, room type, room, booking and payment information without unnecessary redundancy.
- To maintain data integrity using primary keys, foreign keys, NOT NULL constraints and default values.
- To perform INSERT, UPDATE and DELETE operations for registration, check-out, cancellation and payment.
- To retrieve meaningful information using joins, GROUP BY / HAVING, aggregate functions, sub-queries and views.
- To document the project and publish it on GitHub.

3. Modules, Tools and Technologies Used

3.1 Modules

No.	Module	Table(s)	Functions
1	Guest Management	guests	Register guests, store contact details and city, identify repeat guests, remove unused records.
2	Room Management	room_types, rooms	Define room categories with tariff and capacity; track whether each room is Available or Occupied.
3	Booking Management	bookings	Reserve a room for given dates, store nights and total amount, move a booking through Confirmed, Checked-In, Checked-Out or Cancelled.
4	Payment and Billing	payments	Record advance and final payments with the mode of payment; calculate the pending balance of each booking.
5	Reports and Views	booking_summary (view)	Available rooms, booking details, revenue by room type, collection by payment mode, pending dues.

3.2 Tools and Technologies

Component	Tool / Technology
Database (RDBMS)	MySQL
Language	SQL: DDL, DML and DQL (SELECT queries)
Client / Compiler	MySQL Command Line Client (MySQL Workbench can also be used)
Design	Entity Relationship (ER) diagram and relational schema
Version Control	Git and GitHub
Documentation	Microsoft Word

3.3 Database Design

The database contains five entities. One guest can make many bookings, one room can appear in many bookings (on different dates), one room type can have many rooms, and one booking can be paid in more than one instalment. These relationships are shown in the ER diagram below.

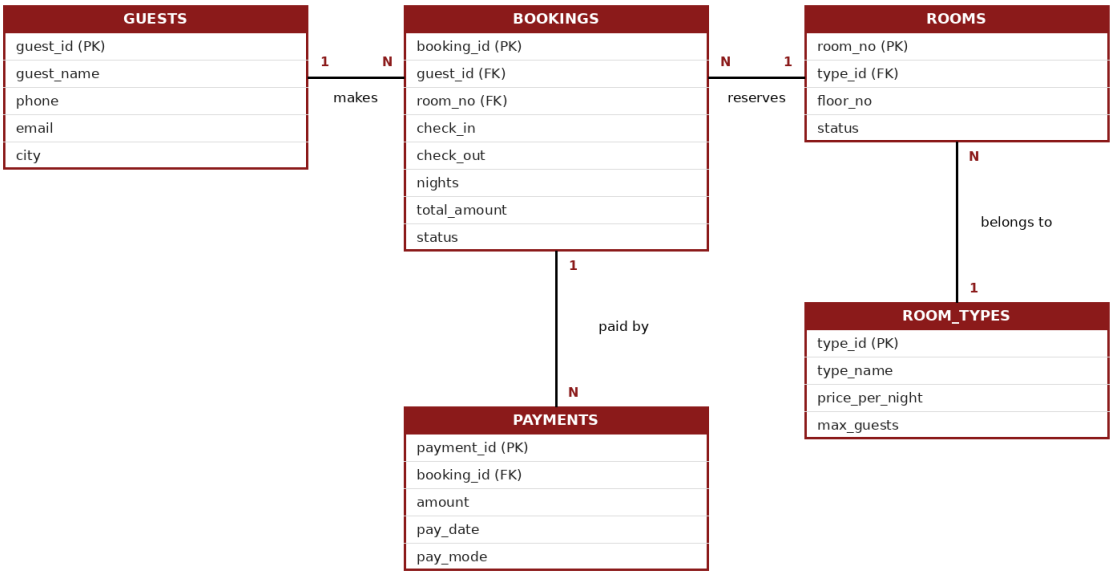


Fig. 1: ER diagram of the Hotel Room Booking System

Table	Primary Key	Foreign Key(s)
guests	guest_id	none
room_types	type_id	none
rooms	room_no	type_id references room_types(type_id)
bookings	booking_id	guest_id references guests(guest_id); room_no references rooms(room_no)
payments	payment_id	booking_id references bookings(booking_id)

3.4 Normalization

All tables are in Third Normal Form (3NF). Every column holds a single atomic value (1NF), every non-key column depends on the whole primary key (2NF), and there is no non-key column that depends on another non-key column (3NF). Room category details such as the tariff are kept only in room_types and not repeated in rooms. The column total_amount in bookings is deliberately stored so that the bill remains correct even if the tariff of a room type changes later.

4. Code with Output

All statements below were run in sequence on the database hotel_booking_db. The code is shown first and the screenshot of its output follows it.

4.1 Database and Table Creation (DDL)

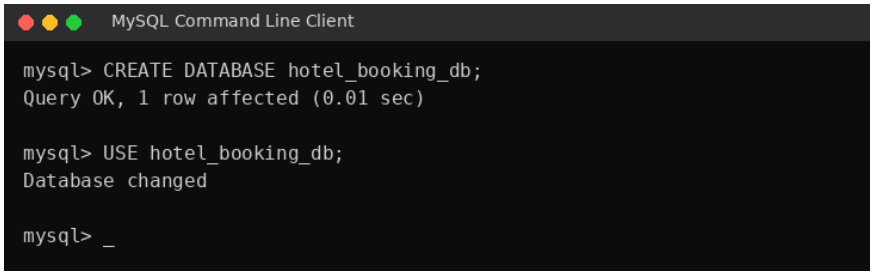
Creating and selecting the database

The database hotel_booking_db is created first and then selected for use.

```
CREATE DATABASE hotel_booking_db;

USE hotel_booking_db;
```

Output:



Creating the tables

Five tables are created with primary keys, foreign keys, NOT NULL constraints and default values.

```
CREATE TABLE guests (
    guest_id INT PRIMARY KEY,
    guest_name VARCHAR(50) NOT NULL,
    phone VARCHAR(10) NOT NULL,
    email VARCHAR(60),
    city VARCHAR(30)
);

CREATE TABLE room_types (
    type_id INT PRIMARY KEY,
    type_name VARCHAR(30) NOT NULL,
    price_per_night INT NOT NULL,
    max_guests INT NOT NULL
);

CREATE TABLE rooms (
    room_no INT PRIMARY KEY,
    type_id INT NOT NULL,
    floor_no INT NOT NULL,
    status VARCHAR(15) NOT NULL DEFAULT 'Available',
    FOREIGN KEY (type_id) REFERENCES room_types(type_id)
);

CREATE TABLE bookings (
    booking_id INT PRIMARY KEY,
    guest_id INT NOT NULL,
    room_no INT NOT NULL,
    check_in DATE NOT NULL,
    check_out DATE NOT NULL,
    nights INT NOT NULL,
    total_amount INT NOT NULL,
    status VARCHAR(15) NOT NULL DEFAULT 'Confirmed',
    FOREIGN KEY (guest_id) REFERENCES guests(guest_id),
```

```
    FOREIGN KEY (room_no) REFERENCES rooms(room_no)
);

CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    booking_id INT NOT NULL,
    amount INT NOT NULL,
    pay_date DATE NOT NULL,
    pay_mode VARCHAR(10) NOT NULL,
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
);
```

Output:

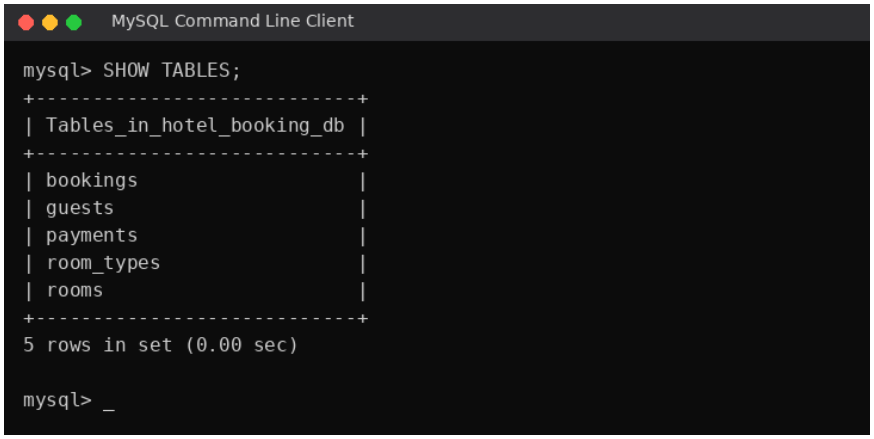


Listing the tables

SHOW TABLES confirms that all five tables exist.

```
SHOW TABLES;
```

Output:



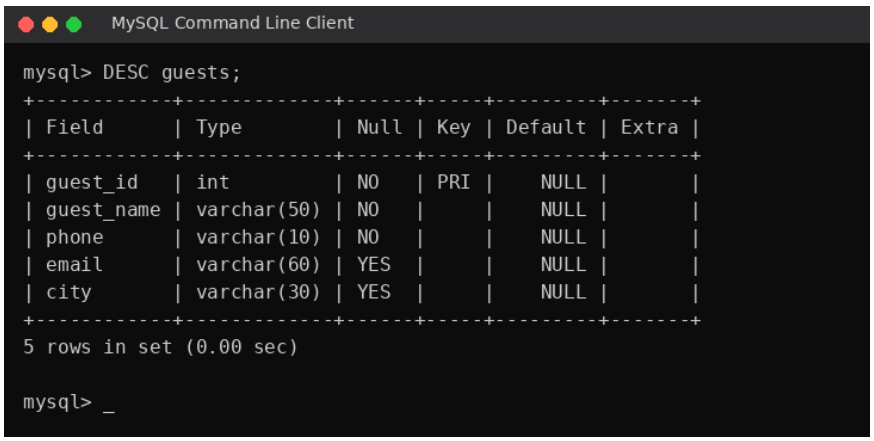
4.2 Table Structures

Structure of the guests table

DESC displays the columns, data types, keys and defaults of guests.

```
DESC guests;
```

Output:

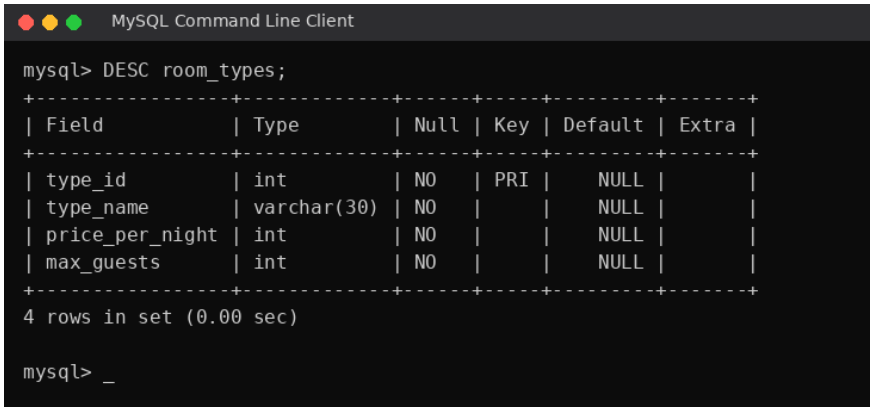


Structure of the room_types table

DESC displays the columns, data types, keys and defaults of room_types.

```
DESC room_types;
```

Output:

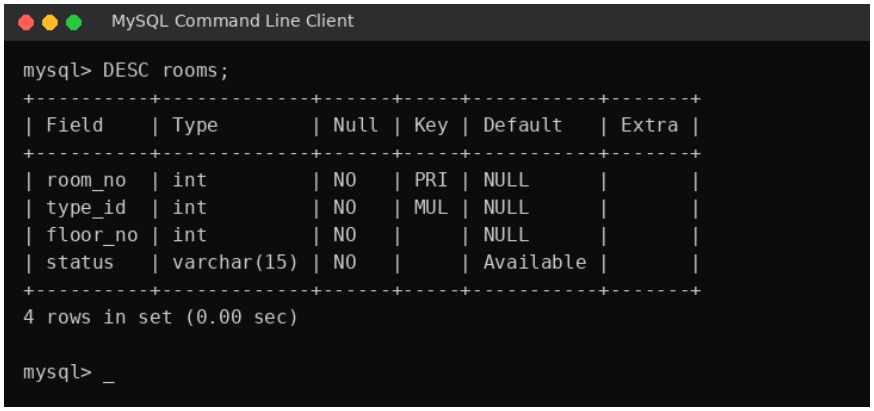


Structure of the rooms table

DESC displays the columns, data types, keys and defaults of rooms.

```
DESC rooms;
```

Output:

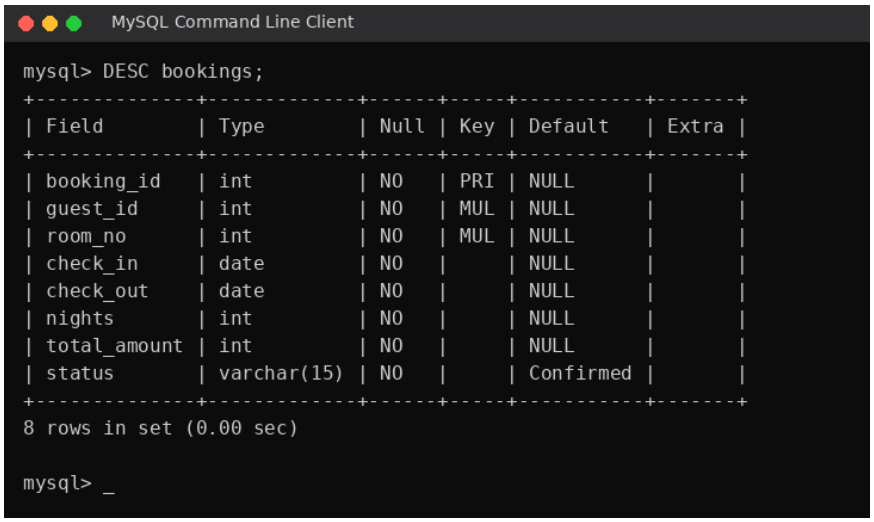


Structure of the bookings table

DESC displays the columns, data types, keys and defaults of bookings.

```
DESC bookings;
```

Output:

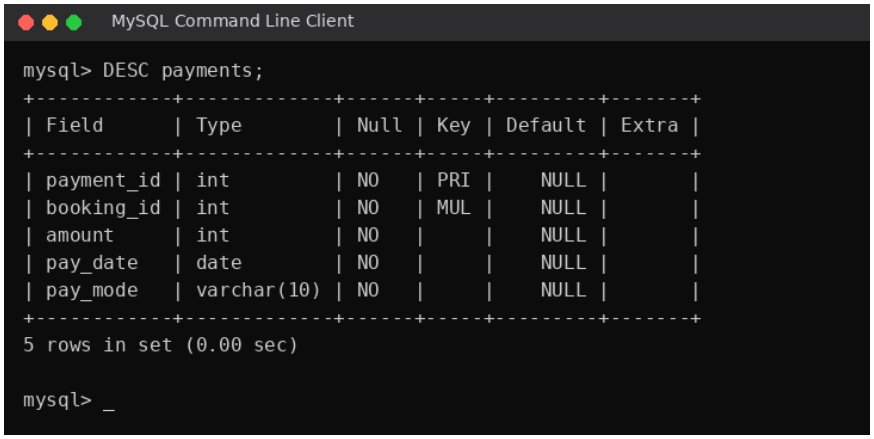


Structure of the payments table

DESC displays the columns, data types, keys and defaults of payments.

```
DESC payments;
```

Output:



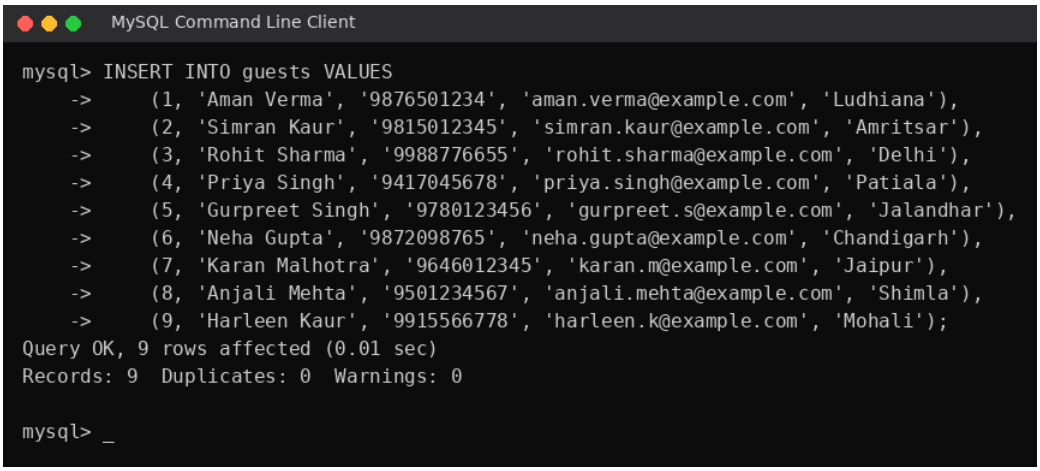
4.3 Inserting Data (DML)

Inserting records into guests

Sample records are inserted into the guests table (DML).

```
INSERT INTO guests VALUES
    (1, 'Aman Verma', '9876501234', 'aman.verma@example.com', 'Ludhiana'),
    (2, 'Simran Kaur', '9815012345', 'simran.kaur@example.com', 'Amritsar'),
    (3, 'Rohit Sharma', '9988776655', 'rohit.sharma@example.com', 'Delhi'),
    (4, 'Priya Singh', '9417045678', 'priya.singh@example.com', 'Patiala'),
    (5, 'Gurpreet Singh', '9780123456', 'gurpreet.s@example.com', 'Jalandhar'),
    (6, 'Neha Gupta', '9872098765', 'neha.gupta@example.com', 'Chandigarh'),
    (7, 'Karan Malhotra', '9646012345', 'karan.m@example.com', 'Jaipur'),
    (8, 'Anjali Mehta', '9501234567', 'anjali.mehta@example.com', 'Shimla'),
    (9, 'Harleen Kaur', '9915566778', 'harleen.k@example.com', 'Mohali');
```

Output:



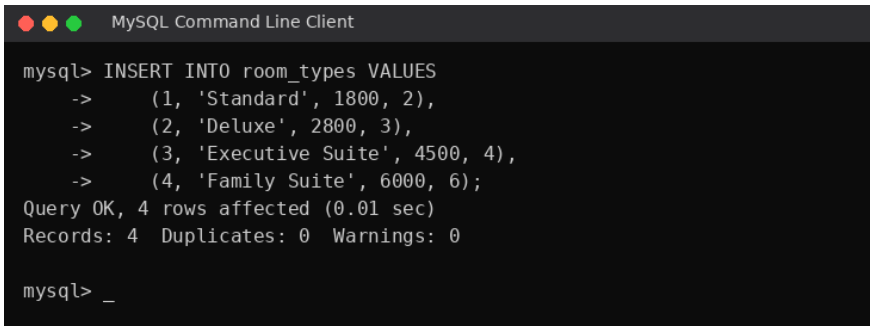
Inserting records into room_types

Sample records are inserted into the room_types table (DML).

```
INSERT INTO room_types VALUES
    (1, 'Standard', 1800, 2),
    (2, 'Deluxe', 2800, 3),
```

```
    (3, 'Executive Suite', 4500, 4),
    (4, 'Family Suite', 6000, 6);
```

Output:

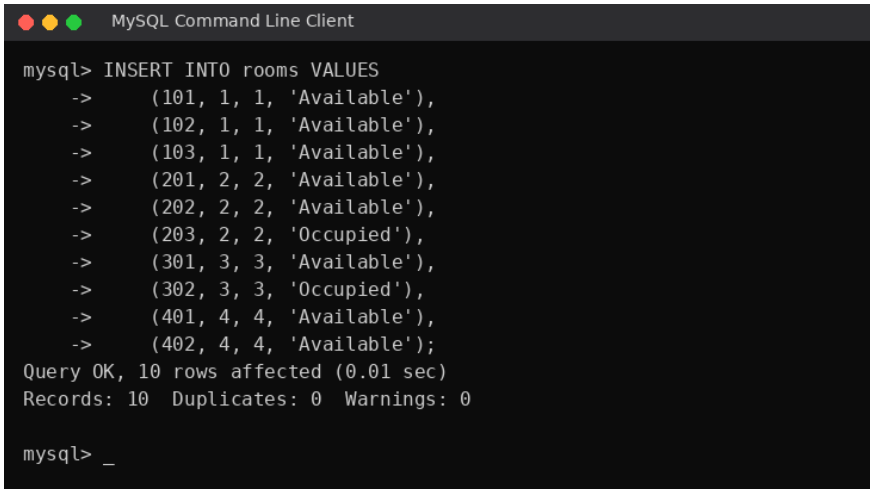


Inserting records into rooms

Sample records are inserted into the rooms table (DML).

```
INSERT INTO rooms VALUES
    (101, 1, 1, 'Available'),
    (102, 1, 1, 'Available'),
    (103, 1, 1, 'Available'),
    (201, 2, 2, 'Available'),
    (202, 2, 2, 'Available'),
    (203, 2, 2, 'Occupied'),
    (301, 3, 3, 'Available'),
    (302, 3, 3, 'Occupied'),
    (401, 4, 4, 'Available'),
    (402, 4, 4, 'Available');
```

Output:

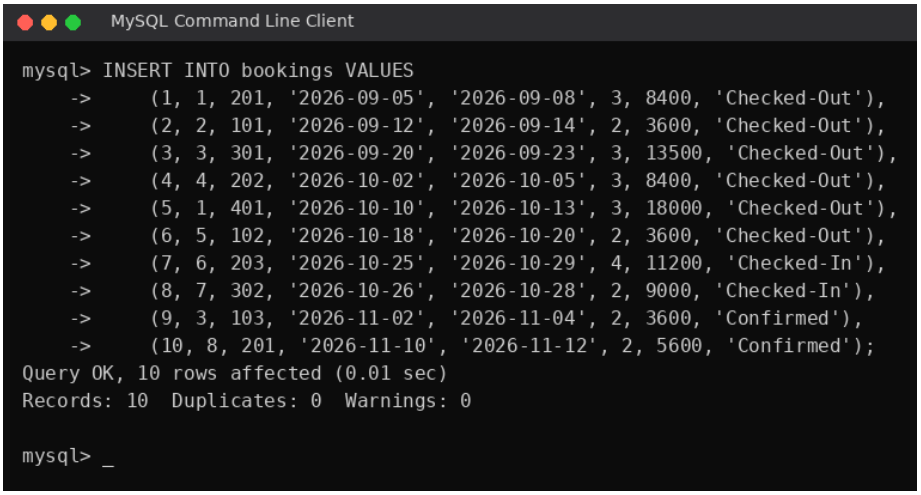


Inserting records into bookings

Sample records are inserted into the bookings table (DML).

```
INSERT INTO bookings VALUES
    (1, 1, 201, '2026-09-05', '2026-09-08', 3, 8400, 'Checked-Out'),
    (2, 2, 101, '2026-09-12', '2026-09-14', 2, 3600, 'Checked-Out'),
    (3, 3, 301, '2026-09-20', '2026-09-23', 3, 13500, 'Checked-Out'),
    (4, 4, 202, '2026-10-02', '2026-10-05', 3, 8400, 'Checked-Out'),
    (5, 1, 401, '2026-10-10', '2026-10-13', 3, 18000, 'Checked-Out'),
    (6, 5, 102, '2026-10-18', '2026-10-20', 2, 3600, 'Checked-Out'),
    (7, 6, 203, '2026-10-25', '2026-10-29', 4, 11200, 'Checked-In'),
    (8, 7, 302, '2026-10-26', '2026-10-28', 2, 9000, 'Checked-In'),
    (9, 3, 103, '2026-11-02', '2026-11-04', 2, 3600, 'Confirmed'),
    (10, 8, 201, '2026-11-10', '2026-11-12', 2, 5600, 'Confirmed');
```

Output:

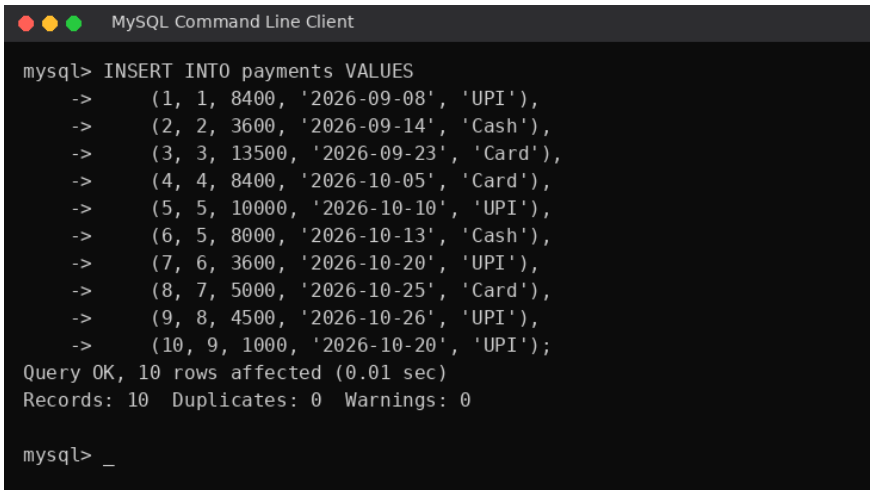


Inserting records into payments

Sample records are inserted into the payments table (DML).

```
INSERT INTO payments VALUES
    (1, 1, 8400, '2026-09-08', 'UPI'),
    (2, 2, 3600, '2026-09-14', 'Cash'),
    (3, 3, 13500, '2026-09-23', 'Card'),
    (4, 4, 8400, '2026-10-05', 'Card'),
    (5, 5, 10000, '2026-10-10', 'UPI'),
    (6, 5, 8000, '2026-10-13', 'Cash'),
    (7, 6, 3600, '2026-10-20', 'UPI'),
    (8, 7, 5000, '2026-10-25', 'Card'),
    (9, 8, 4500, '2026-10-26', 'UPI'),
    (10, 9, 1000, '2026-10-20', 'UPI');
```

Output:



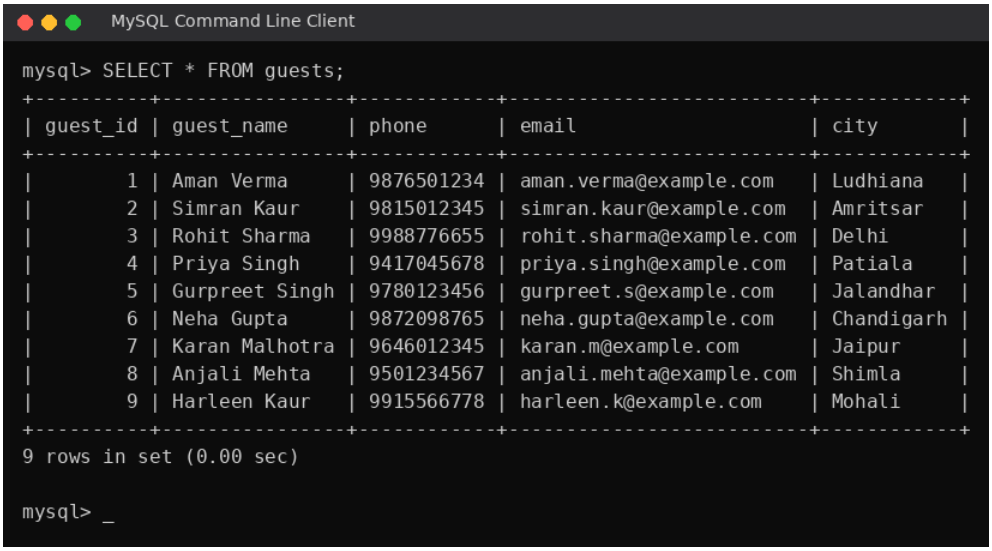
4.4 Viewing the Stored Data

Displaying the guests table

All records of the guests table are displayed.

```
SELECT * FROM guests;
```

Output:

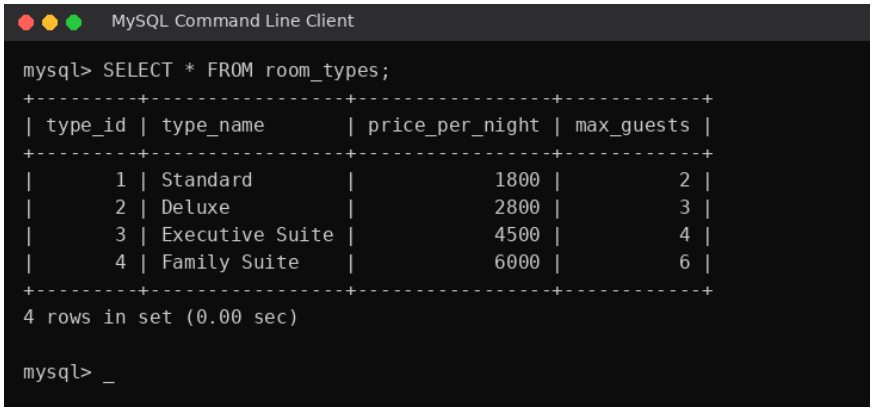


Displaying the room_types table

All records of the room_types table are displayed.

```
SELECT * FROM room_types;
```

Output:

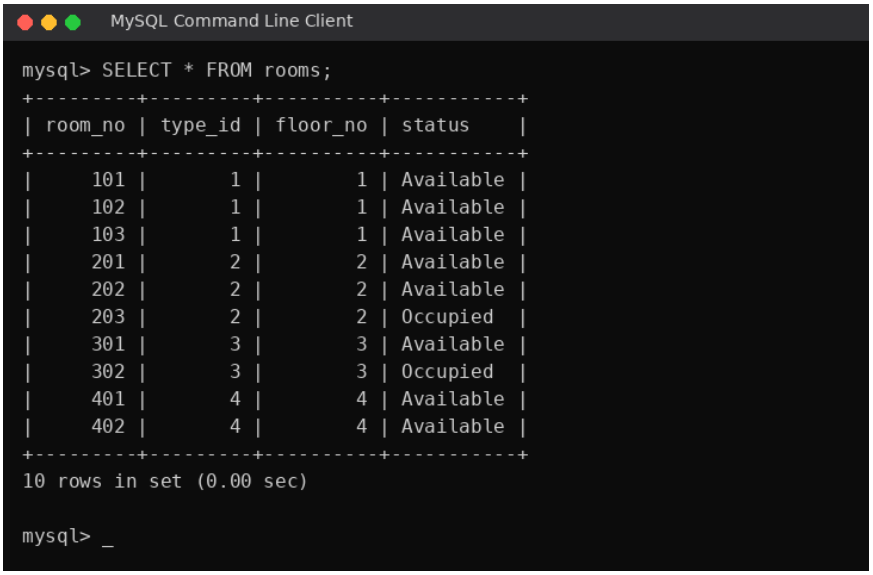


Displaying the rooms table

All records of the rooms table are displayed.

```
SELECT * FROM rooms;
```

Output:

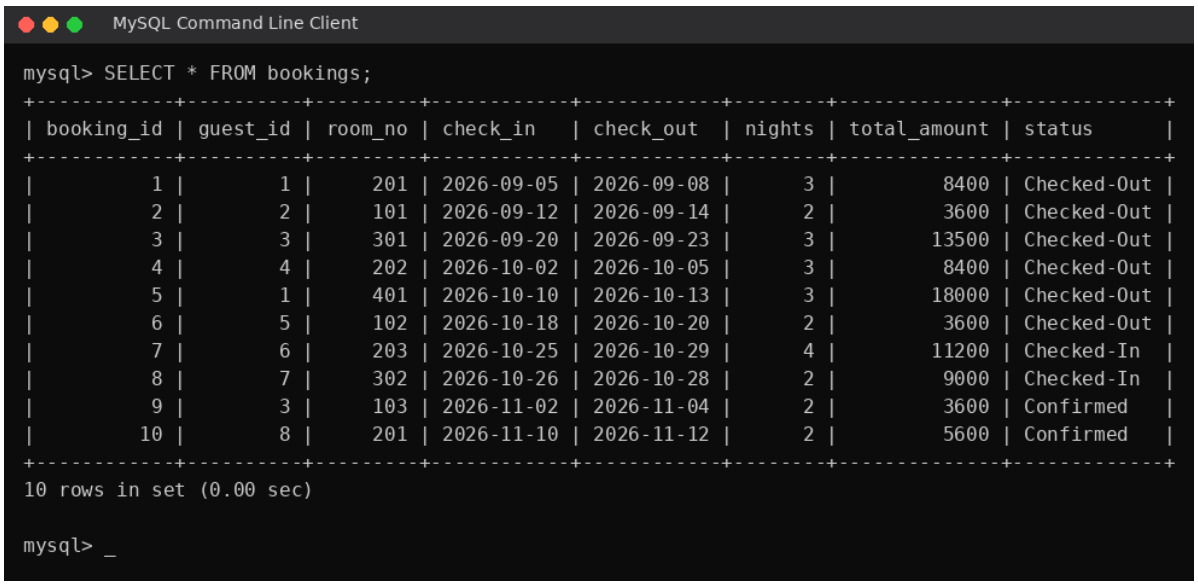


Displaying the bookings table

All records of the bookings table are displayed.

```
SELECT * FROM bookings;
```

Output:

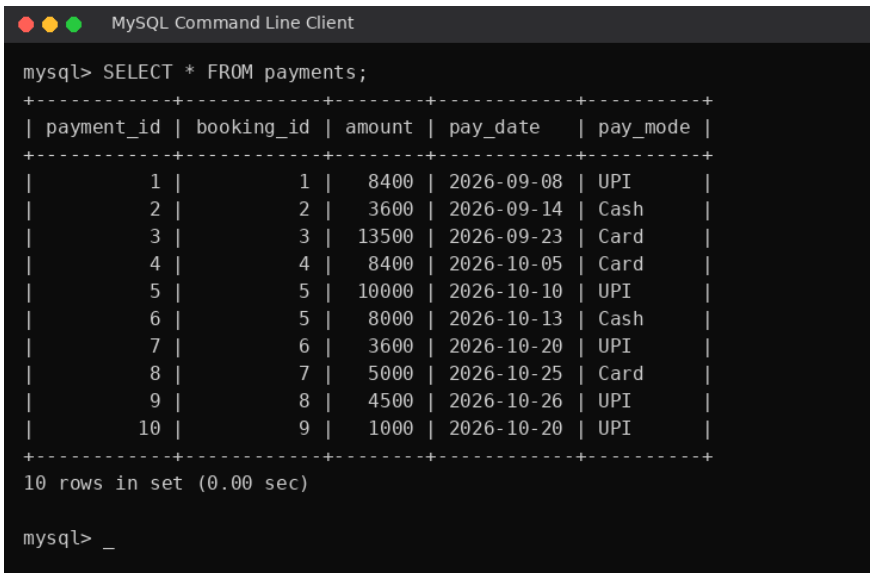


Displaying the payments table

All records of the payments table are displayed.

```
SELECT * FROM payments;
```

Output:



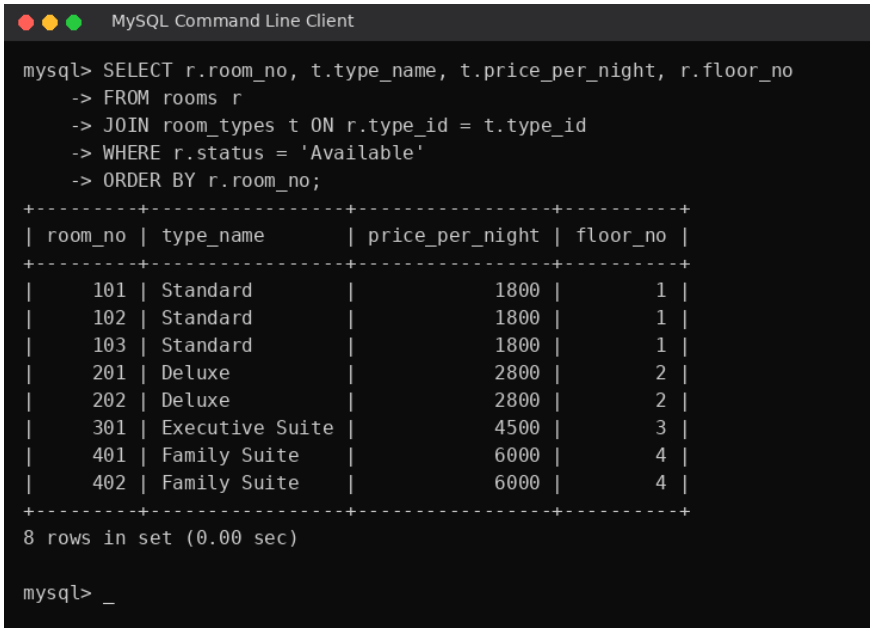
4.5 Queries (DQL)

Query 1: Rooms currently available

Joins rooms with room_types to list every available room with its category and tariff.

```
SELECT r.room_no, t.type_name, t.price_per_night, r.floor_no
FROM rooms r
JOIN room_types t ON r.type_id = t.type_id
WHERE r.status = 'Available'
ORDER BY r.room_no;
```

Output:



Query 2: Complete booking details

A three-table JOIN shows each booking along with the guest name and room number.

```
SELECT b.booking_id, g.guest_name, b.room_no, b.check_in,
       b.check_out, b.total_amount, b.status
FROM bookings b
```

```
JOIN guests g ON b.guest_id = g.guest_id
ORDER BY b.booking_id;
```

Output:

MySQL Command Line Client

```
mysql> SELECT b.booking_id, g.guest_name, b.room_no, b.check_in,
    ->        b.check_out, b.total_amount, b.status
    -> FROM bookings b
    -> JOIN guests g ON b.guest_id = g.guest_id
    -> ORDER BY b.booking_id;
+------------+---------------+---------+------------+------------+--------------+-------------+
| booking_id | guest_name    | room_no | check_in   | check_out  | total_amount | status      |
+------------+---------------+---------+------------+------------+--------------+-------------+
|          1 | Aman Verma    |     201 | 2026-09-05 | 2026-09-08 |         8400 | Checked-Out |
|          2 | Simran Kaur   |     101 | 2026-09-12 | 2026-09-14 |         3600 | Checked-Out |
|          3 | Rohit Sharma  |     301 | 2026-09-20 | 2026-09-23 |        13500 | Checked-Out |
|          4 | Priya Singh   |     202 | 2026-10-02 | 2026-10-05 |         8400 | Checked-Out |
|          5 | Aman Verma    |     401 | 2026-10-10 | 2026-10-13 |        18000 | Checked-Out |
|          6 | Gurpreet Singh|     102 | 2026-10-18 | 2026-10-20 |         3600 | Checked-Out |
|          7 | Neha Gupta    |     203 | 2026-10-25 | 2026-10-29 |        11200 | Checked-In  |
|          8 | Karan Malhotra|     302 | 2026-10-26 | 2026-10-28 |         9000 | Checked-In  |
|          9 | Rohit Sharma  |     103 | 2026-11-02 | 2026-11-04 |         3600 | Confirmed   |
|         10 | Anjali Mehta  |     201 | 2026-11-10 | 2026-11-12 |         5600 | Confirmed   |
+------------+---------------+---------+------------+------------+--------------+-------------+
10 rows in set (0.00 sec)

mysql> _
```

Query 3: Revenue by room type

GROUP BY with COUNT and SUM calculates the bookings and income of every room category (cancelled bookings excluded).

```
SELECT t.type_name, COUNT(b.booking_id) AS total_bookings,
       SUM(b.total_amount) AS total_revenue
FROM bookings b
JOIN rooms r ON b.room_no = r.room_no
JOIN room_types t ON r.type_id = t.type_id
WHERE b.status <> 'Cancelled'
GROUP BY t.type_name
ORDER BY total_revenue DESC;
```

Output:

MySQL Command Line Client

```
mysql> SELECT t.type_name, COUNT(b.booking_id) AS total_bookings,
    ->        SUM(b.total_amount) AS total_revenue
    -> FROM bookings b
    -> JOIN rooms r ON b.room_no = r.room_no
    -> JOIN room_types t ON r.type_id = t.type_id
    -> WHERE b.status <> 'Cancelled'
    -> GROUP BY t.type_name
    -> ORDER BY total_revenue DESC;
+----------------+----------------+---------------+
| type_name      | total_bookings | total_revenue |
+----------------+----------------+---------------+
| Deluxe         |              4 |         33600 |
| Executive Suite|              2 |         22500 |
| Family Suite   |              1 |         18000 |
| Standard       |              3 |         10800 |
+----------------+----------------+---------------+
4 rows in set (0.00 sec)

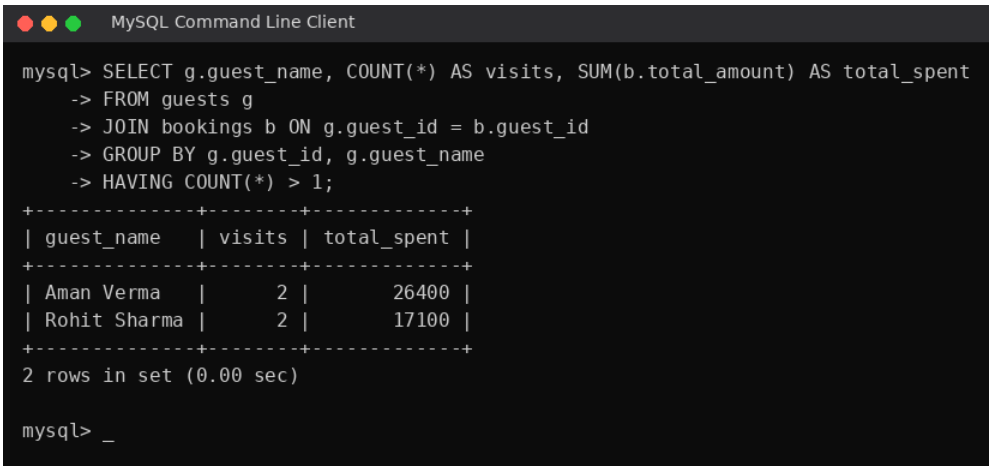
mysql> _
```

Query 4: Repeat guests

HAVING filters the grouped result to guests who have booked more than once.

```
SELECT g.guest_name, COUNT(*) AS visits, SUM(b.total_amount) AS total_spent
FROM guests g
JOIN bookings b ON g.guest_id = b.guest_id
GROUP BY g.guest_id, g.guest_name
HAVING COUNT(*) > 1;
```

Output:

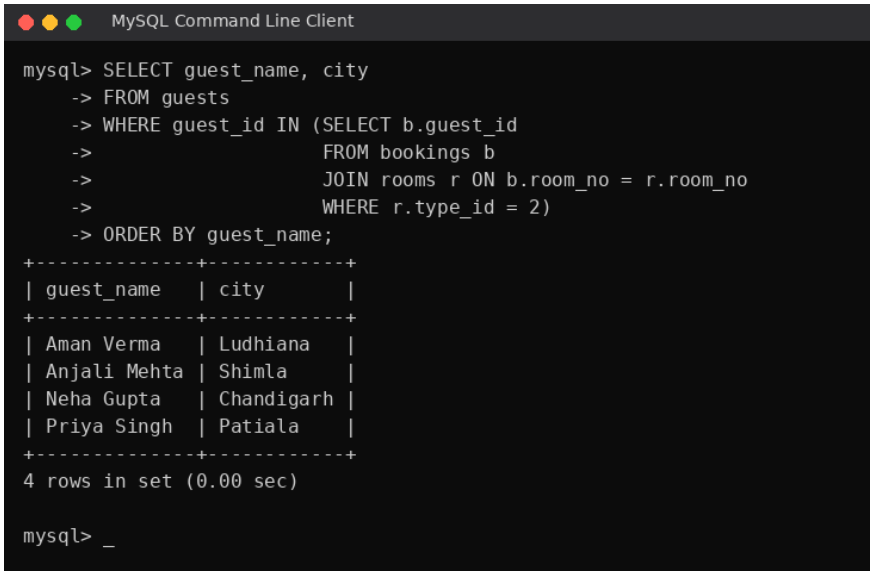


Query 5: Guests who booked a Deluxe room

A nested sub-query finds the guests whose bookings belong to room type 2 (Deluxe).

```
SELECT guest_name, city
FROM guests
WHERE guest_id IN (SELECT b.guest_id
                   FROM bookings b
                   JOIN rooms r ON b.room_no = r.room_no
                   WHERE r.type_id = 2)
ORDER BY guest_name;
```

Output:

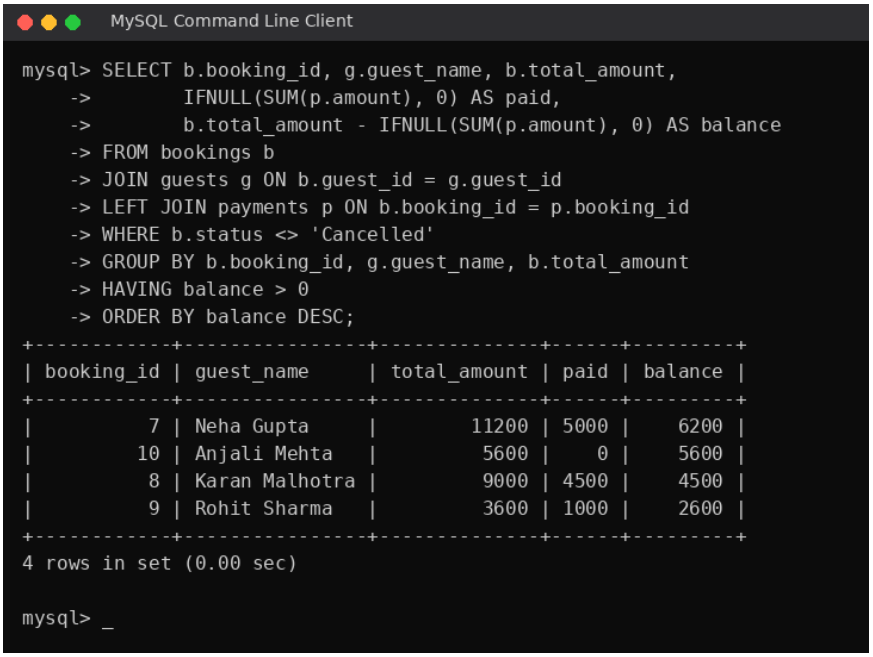


Query 6: Pending payments

A LEFT JOIN with aggregation finds bookings whose payments are still incomplete.

```
SELECT b.booking_id, g.guest_name, b.total_amount,
       IFNULL(SUM(p.amount), 0) AS paid,
       b.total_amount - IFNULL(SUM(p.amount), 0) AS balance
FROM bookings b
JOIN guests g ON b.guest_id = g.guest_id
LEFT JOIN payments p ON b.booking_id = p.booking_id
WHERE b.status <> 'Cancelled'
GROUP BY b.booking_id, g.guest_name, b.total_amount
HAVING balance > 0
ORDER BY balance DESC;
```

Output:

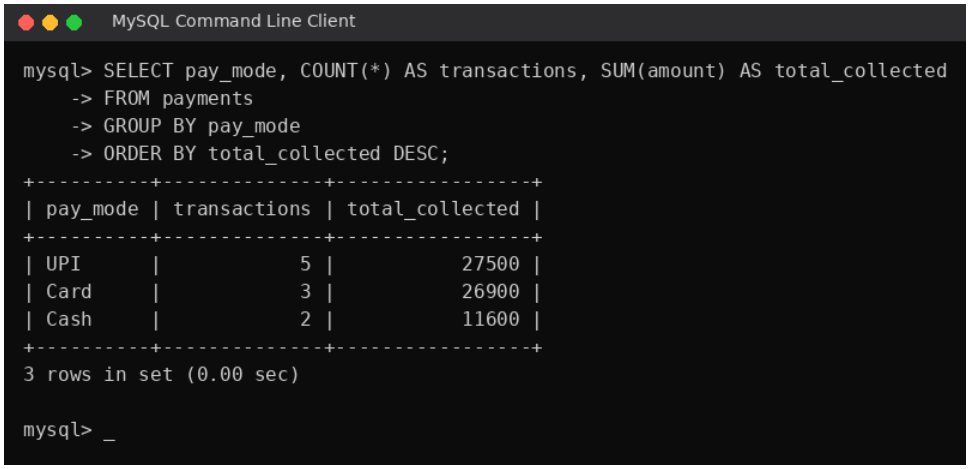


Query 7: Collection by payment mode

Summarises the number of transactions and the amount collected for each payment mode.

```
SELECT pay_mode, COUNT(*) AS transactions, SUM(amount) AS total_collected
FROM payments
GROUP BY pay_mode
ORDER BY total_collected DESC;
```

Output:

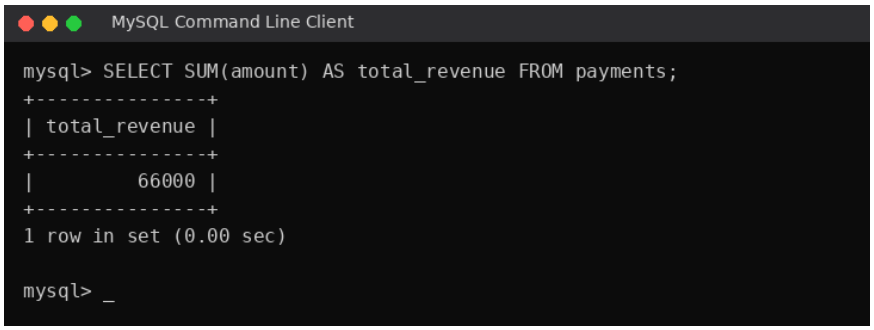


Query 8: Total revenue collected

An aggregate function returns the total amount received so far.

```
SELECT SUM(amount) AS total_revenue FROM payments;
```

Output:

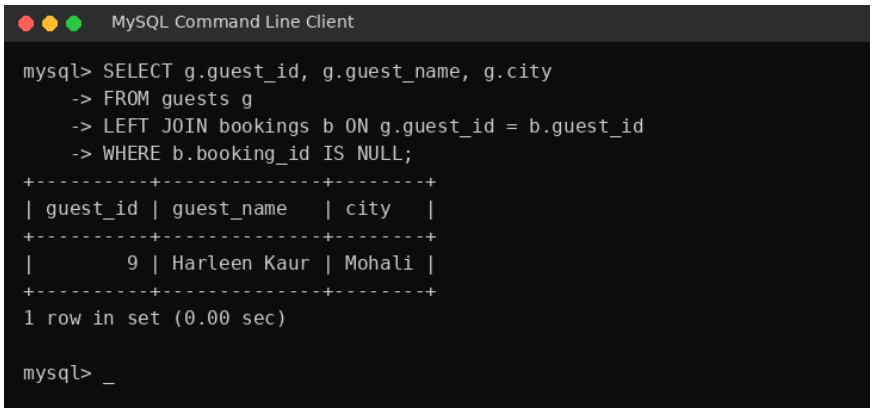


Query 9: Guests who have never booked

A LEFT JOIN with IS NULL lists registered guests having no booking.

```
SELECT g.guest_id, g.guest_name, g.city
FROM guests g
LEFT JOIN bookings b ON g.guest_id = b.guest_id
WHERE b.booking_id IS NULL;
```

Output:



4.6 Update and Delete Operations

Check-out and final payment (UPDATE and INSERT)

When guest Karan Malhotra checks out, the booking is closed, the room is released and the balance payment is recorded.

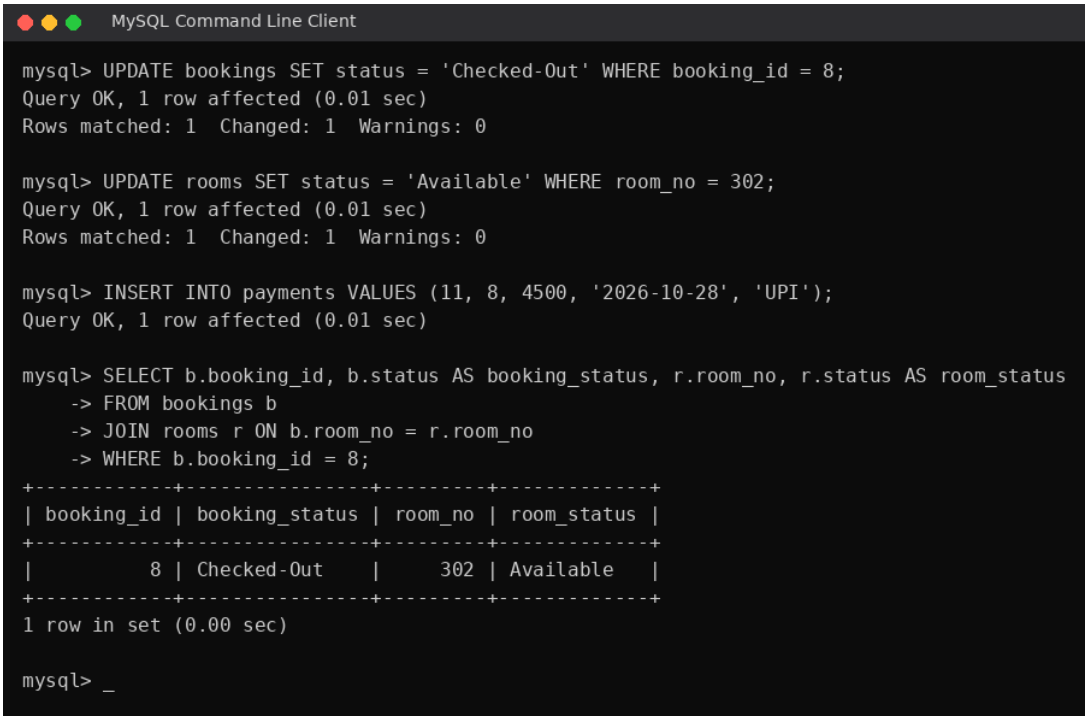
```
UPDATE bookings SET status = 'Checked-Out' WHERE booking_id = 8;

UPDATE rooms SET status = 'Available' WHERE room_no = 302;

INSERT INTO payments VALUES (11, 8, 4500, '2026-10-28', 'UPI');

SELECT b.booking_id, b.status AS booking_status, r.room_no, r.status AS room_status
FROM bookings b
JOIN rooms r ON b.room_no = r.room_no
WHERE b.booking_id = 8;
```

Output:



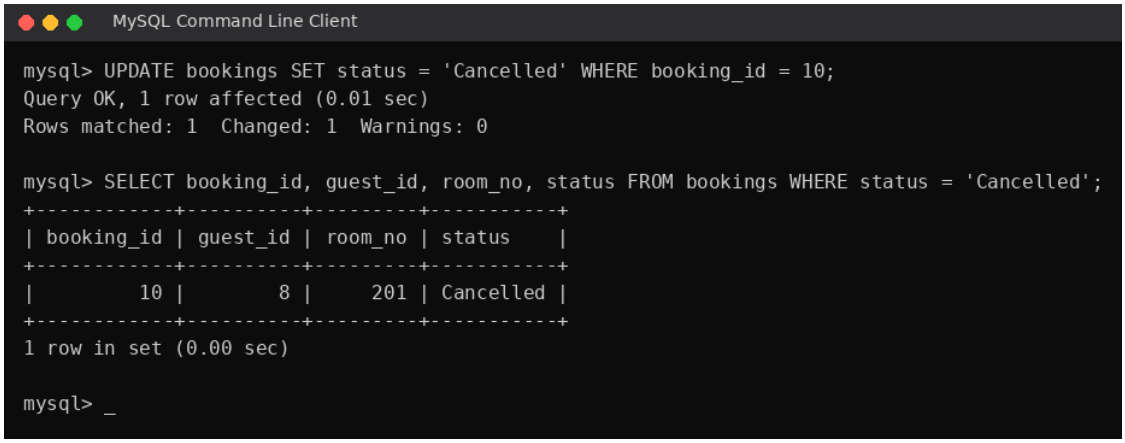
Cancelling a booking (UPDATE)

Booking 10 is cancelled by the guest. The status is changed instead of deleting the record so that the history is preserved.

```
UPDATE bookings SET status = 'Cancelled' WHERE booking_id = 10;

SELECT booking_id, guest_id, room_no, status FROM bookings WHERE status =
'Cancelled';
```

Output:



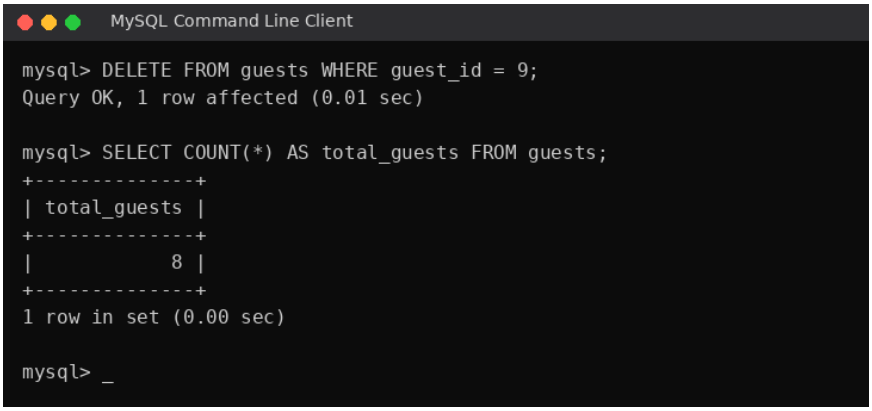
Deleting an unused guest record (DELETE)

Guest 9 has no bookings (Query 9), so the record is removed safely without violating any foreign key.

```
DELETE FROM guests WHERE guest_id = 9;

SELECT COUNT(*) AS total_guests FROM guests;
```

Output:



4.7 View

Creating and using a view

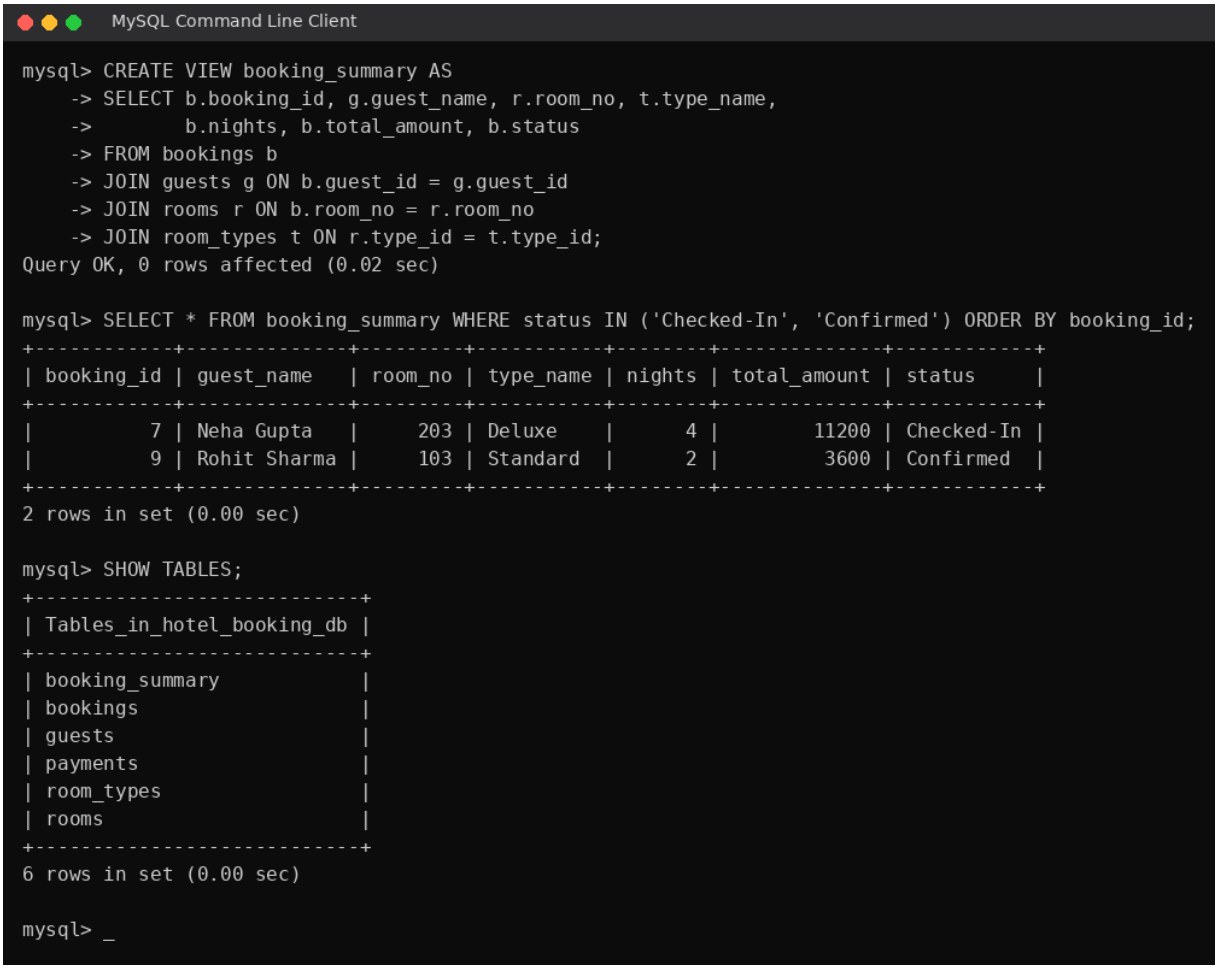
A view stores the commonly used four-table join so that it can be queried like a table.

```
CREATE VIEW booking_summary AS
SELECT b.booking_id, g.guest_name, r.room_no, t.type_name,
       b.nights, b.total_amount, b.status
FROM bookings b
JOIN guests g ON b.guest_id = g.guest_id
JOIN rooms r ON b.room_no = r.room_no
JOIN room_types t ON r.type_id = t.type_id;

SELECT * FROM booking_summary WHERE status IN ('Checked-In', 'Confirmed') ORDER BY
booking_id;

SHOW TABLES;
```

Output:



5. Conclusion

The Hotel Room Booking System shows how a relational database can replace manual registers in a hotel. The five tables are linked through primary and foreign keys, so a booking can never refer to a guest or room that does not exist, and a guest with bookings cannot be deleted by mistake. Using SQL, the system answers practical questions instantly: the list of available rooms, the full booking record with guest names, revenue of every room category, repeat customers and the balance still to be collected from each guest.

While doing this project I practised designing an ER diagram, converting it into tables, applying constraints, and writing queries with joins, aggregate functions, sub-queries and views. The project also showed why storing each fact in only one place (normalization) makes updates such as check-out and cancellation simple and safe.

Future Scope

- Adding a front-end (web or desktop) application connected to the database.
- Preventing overlapping dates for the same room using a trigger or stored procedure.
- Adding staff, services (food, laundry) and feedback tables, with GST in billing.

- Generating automatic invoices and monthly occupancy reports.

GitHub Repository

https://github.com/Randeepsingh885/Hotel-Room-Booking-System-DBMS-Mini-Project-.git